



**PES**

**UNIVERSITY**

CELEBRATING 50 YEARS

# PROGRAMMING WITH PYTHON

---

**Lekha A**

Computer Applications

# PROGRAMMING WITH PYTHON

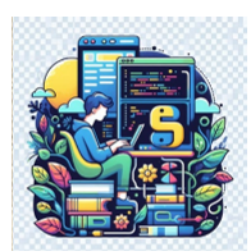
---

## Object-Oriented Programming (OOP) and Advanced Concepts

### Encapsulation - II

**Lekha A**

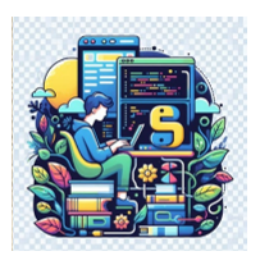
Computer Applications



# PROGRAMMING WITH PYTHON

## Getters and setters

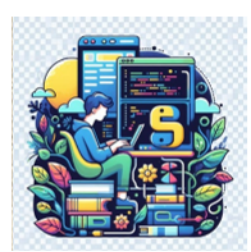
- The primary purpose of using getters and setters in object-oriented programs is to ensure data encapsulation.
- Use the getter method to access data members and the setter methods to modify the data members.



# PROGRAMMING WITH PYTHON

## Getters and setters

- In Python, private variables are not hidden fields like in other programming languages.
- The getters and setters methods are often used when
  - When there is a need to avoid direct access to private variables
  - To add validation logic for setting a value



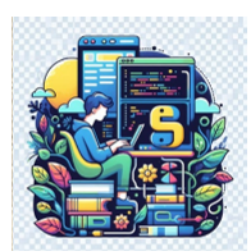
# PROGRAMMING WITH PYTHON

## Getters and setters

```
class Student:
    def __init__(self, name, age):
        # private member
        self.name = name
        self.__age = age

    # getter method
    def get_age(self):
        return self.__age

    # setter method
    def set_age(self, age):
        self.__age = age
```



# PROGRAMMING WITH PYTHON

## Getters and setters

- `stud = Student('Dhruv', 14)`

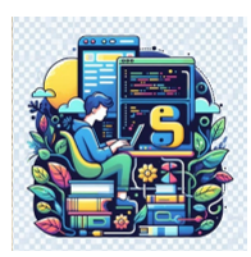
```
# retrieving age using getter  
print('Name:', stud.name, stud.get_age())
```

Name: Dhruv 14

```
# changing age using setter  
stud.set_age(16)
```

```
# retrieving age using getter  
print('Name:', stud.name, stud.get_age())
```

Name: Dhruv 16

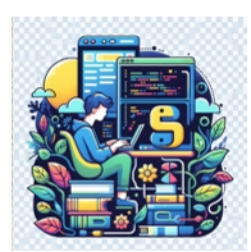


# PROGRAMMING WITH PYTHON

## Getters and setters

---

- Use encapsulation to implement information hiding and apply additional validation before changing the values of the object attributes (data member).



# PROGRAMMING WITH PYTHON

## Getters and setters

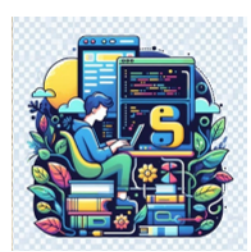
```
class Student:
    def __init__(self, name, roll_no, age):
        # private member
        self.name = name
        # private members to restrict access
        # avoid direct data modification
        self.__roll_no = roll_no
        self.__age = age

    def show(self):
        print('Student Details:', self.name, self.__roll_no)

    # getter methods
    def get_roll_no(self):
        return self.__roll_no

    # setter method to modify data member
    # condition to allow data modification with rules
    def set_roll_no(self, number):
        if number > 50:
            print('Invalid roll no. Please set correct roll number')
        else:
            self.__roll_no = number
```





# PROGRAMMING WITH PYTHON

## Getters and setters

- ```
std1 = Student('Sachi', 10, 15)
```

```
# before Modify  
std1.show()
```

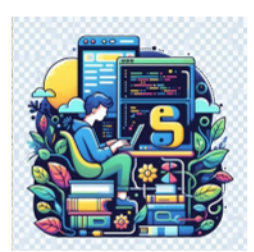
Student Details: Sachi 10

```
# changing roll number using setter  
std1.set_roll_no(120)
```

Invalid roll no. Please set correct roll number

```
std1.set_roll_no(25)  
std1.show()
```

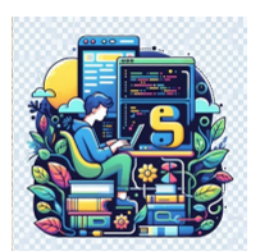
Student Details: Sachi 25



# PROGRAMMING WITH PYTHON

## Encapsulation - Advantages

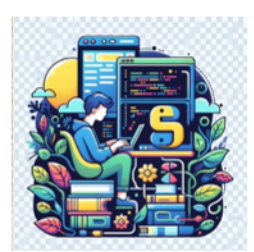
- Security
  - The main advantage of using encapsulation is the security of the data.
  - Encapsulation protects an object from unauthorized access.
  - It allows private and protected access levels to prevent any accidental data modification.



# PROGRAMMING WITH PYTHON

## Encapsulation - Advantages

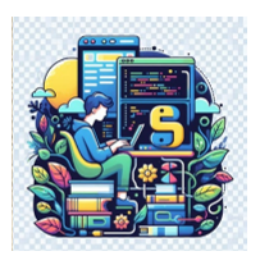
- Data Hiding
  - The user would not be knowing what is going on behind the scene.
  - They would only be knowing that to modify a data member, there is a need to call the setter method.
  - To read a data member, there is a need to call the getter method.
    - What these setter and getter methods are doing is hidden from them.



# PROGRAMMING WITH PYTHON

## Encapsulation - Advantages

- Simplicity
  - It simplifies the maintenance of the application by keeping classes separated and preventing them from tightly coupling with each other.
- Aesthetics
  - Bundling data and methods within a class makes code more readable and maintainable

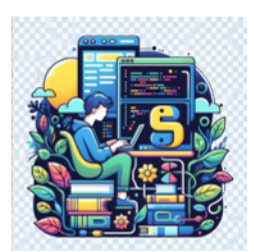


# PROGRAMMING WITH PYTHON

## Encapsulation - Advantages

---

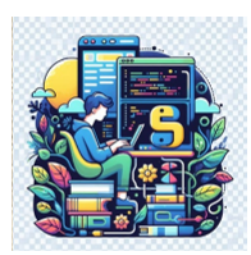
- Reusability
  - It is easier to reuse code when it is encapsulated.
  - Creating objects that contain common functionality allows the reuse in many settings.



# PROGRAMMING WITH PYTHON

## Encapsulation - Advantages

- There is no need to know the architecture of the methods and the data and can just focus on making use of these functional, encapsulated units for our applications.
- This results in a more organized and clean code.
- The user experience also improves greatly and makes it easier to understand applications as a whole.



# PROGRAMMING WITH PYTHON

## Recap

---

- Getters and Setters in Python
- Advantages



**THANK YOU**

---

**Lekha A**  
Computer Applications